Hoja de trabajo 2 — Transformers y mecanismos de atención

CC3092 Deep Learning y Sistemas Inteligentes · Ian Cumes · Universidad del Valle de Guatemala

1. De las RNN a la atención

Cuello de botella seq2seq. En el encoder-decoder con RNN el encoder comprime toda la oración de entrada en un único vector de tamaño fijo (su último estado oculto) y el decoder debe generar la salida solo a partir de él. Con oraciones largas, la información de los primeros tokens se diluye o se sobrescribe y la calidad (p. ej. BLEU) cae al crecer la longitud. **Bahdanau et al. (2015)** conservan *todos* los estados del encoder y, en cada paso del decoder, calculan un vector de contexto distinto como promedio ponderado de ellos; los pesos (alineamiento suave) dependen del estado actual del decoder, de modo que el modelo «vuelve a mirar» la parte relevante de la entrada.

Aditiva vs multiplicativa. Bahdanau puntúa con una pequeña red: $e = v^T \tanh(W_1 s + W_2 h)$. Luong (2015) usa un producto punto ($s^T h$) o bilineal ($s^T W h$). Ambas tienen complejidad asintótica similar, pero el producto punto se expresa como **una sola multiplicación de matrices** QK^T, operación altamente optimizada en GPU/TPU (GEMM, tensor cores), sin la no linealidad elemento a elemento ni el tensor intermedio de la aditiva; por eso es más eficiente en hardware moderno (la aditiva solo rendía mejor con d_k grande sin escalar, problema que corrige el factor $1/\sqrt{d_k}$).

Q, K, V como diccionario suave. En un diccionario clásico, una consulta coincide exactamente con una llave y devuelve su valor. En la atención, la consulta se compara con *todas* las llaves (similitud q·k), la softmax convierte las similitudes en pesos que suman 1 y se devuelve la mezcla ponderada de los valores: una búsqueda diferenciable. En **self-attention** Q, K y V son proyecciones lineales (W_Q, W_K, W_V) de la *misma* secuencia; en **cross-attention** Q sale del decoder y K, V de la salida del encoder.

Limitaciones de las RNN que elimina el Transformer. (i) El cómputo secuencial $h_t = f(h_{t-1}, x_t)$ impide paralelizar a lo largo del tiempo; la atención procesa todos los tokens a la vez. (ii) Dos tokens a distancia n están separados por O(n) pasos en una RNN y por O(1) en la atención, lo que facilita las dependencias de largo alcance. (iii) El gradiente ya no atraviesa n multiplicaciones por la misma matriz recurrente, lo que reduce el desvanecimiento/explosión (junto con las conexiones residuales).

2. La arquitectura Transformer

Scaled dot-product attention. Si q_i, k_i son independientes con media 0 y varianza 1, $q \cdot k = \Sigma\, q_i k_i$ es suma de d_k términos de media 0 y varianza 1, así que **Var(q·k) = d_k**. Los logits crecen como $\sqrt{d_k}$, la softmax se satura (casi one-hot) y su jacobiano $\mathrm{diag}(p) - pp^T$ tiende a 0: los gradientes se desvanecen. Dividir entre $\sqrt{d_k}$ devuelve la varianza a 1. Lo verificamos numéricamente (fig. 1): con d_k=1024 la varianza sin escalar es 1028 (escalada 1.00), la probabilidad máxima promedio sube a 0.96 (vs 0.25) y la norma del jacobiano cae a 0.052 (vs 0.305, constante con escalamiento).

Multi-head attention. Con h cabezas de dimensión d_{model}/h cada cabeza aprende su propio subespacio y su propio patrón de atención (sintaxis, posición, correferencia…); una sola cabeza promediaría todo en una única distribución. El número de parámetros **no cambia** ($4 \cdot d_{model}^2$ + sesgos): para d_{model}=512 obtuvimos 1,050,624 parámetros con h = 1, 2, 4 u 8; el costo de cómputo también es prácticamente el mismo.

Codificación posicional. La self-attention es equivariante a permutaciones: si se permutan los tokens, la salida se permuta igual, por lo que no distingue «perro muerde hombre» de «hombre muerde perro». El artículo suma $PE(pos,2i) = \sin(pos/10000^{2i/d})$ y $PE(pos,2i+1) = \cos(\cdot)$: frecuencias geométricas, sin parámetros, y PE(pos+k) es función lineal de PE(pos), lo que facilita atender por posición relativa. Los **embeddings aprendidos** (BERT, GPT-2) son una tabla entrenable: flexibles pero no extrapolan más allá de la longitud máxima vista. **RoPE** (Su et al., 2021) rota q y k según su posición, de modo que q·k depende solo de la distancia relativa; se aplica en cada capa y es el estándar actual (LLaMA, Mistral), con mejor extrapolación vía interpolación de frecuencias.

Bloque completo. Cada subcapa (atención y FFN) va envuelta en una conexión residual x + Sublayer(x), que crea un camino directo para el gradiente, y en LayerNorm, que estabiliza la escala de las activaciones. La **FFN por posición** (dos lineales con ReLU/GELU, $d_{ff} \approx 4 \cdot d_{model}$) se aplica a cada token por separado y aporta la mayor parte de los parámetros y la no linealidad. **Post-LN** (original): LN(x + Sublayer(x)); el gradiente pasa por LN en cada capa, requiere *warm-up* y es inestable en modelos profundos. **Pre-LN**: x + Sublayer(LN(x)); la ruta residual queda limpia, entrena estable sin warm-up y escala a cientos de capas (Xiong et al., 2020), a costa de un rendimiento final a veces ligeramente menor.

Complejidad. Tiempo $O(n^2 \cdot d)$ y memoria $O(n^2)$ por la matriz QK^T. En nuestra medición (fig. 2) la pendiente log-log del tiempo fue 1.60 (≈2) y la matriz de atención pasa de 0.5 MB (n=128) a 512 MB (n=4096) por capa. Alternativas: **FlashAttention** (Dao et al., 2022) calcula la atención exacta por bloques en SRAM sin materializar n×n: memoria O(n), mismo resultado, pero el tiempo sigue siendo $O(n^2)$ y requiere kernels específicos de hardware. **Atención por ventanas** (Longformer, Mistral) restringe cada token a w vecinos (+ tokens globales): O(n·w), sacrificando interacción directa a larga distancia. **Atención lineal** (Katharopoulos et al., 2020) reemplaza la softmax por un kernel $\phi(q)\phi(k)^T$ y reordena el producto: O(n), pero aproxima la atención y suele perder precisión en tareas de recuperación exacta.

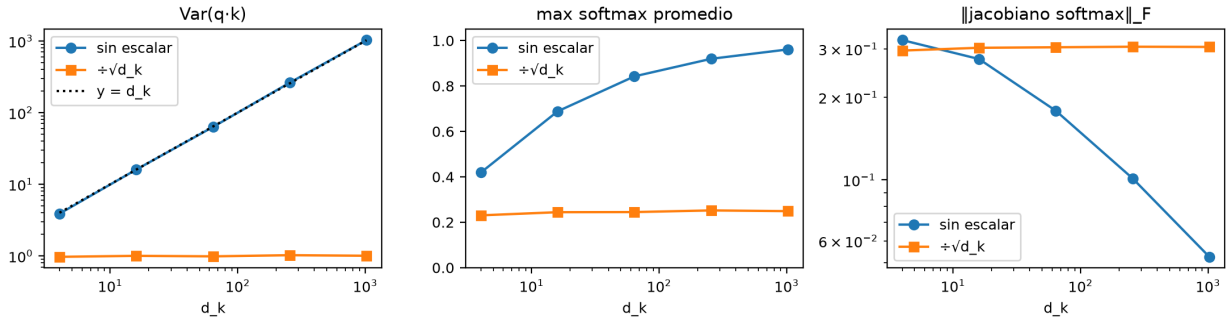


Fig. 1. Efecto de d_k con q, k ~ N(0, I) y 16 llaves: varianza de q·k, probabilidad máxima de la softmax y norma del jacobiano, con y sin el factor $1/\sqrt{d_k}$.

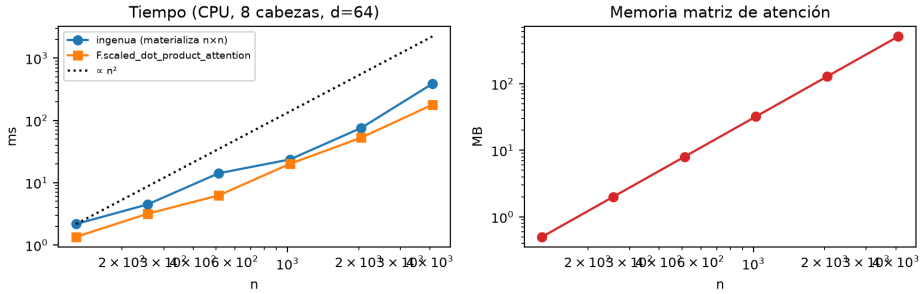


Fig. 2. Costo de la self-attention en CPU (8 cabezas, d=64): tiempo de la versión ingenua vs `F.scaled_dot_product_attention` y memoria de la matriz n×n.

3. Atención en PyTorch y Hugging Face

F.scaled_dot_product_attention(q, k, v, attn_mask, dropout_p, is_causal, scale): atención fusionada que elige automáticamente un kernel (FlashAttention, memory-efficient o matemático). *attn_mask* es booleana (True = se puede atender) o aditiva (float, –inf bloquea); *dropout_p* aplica dropout a los pesos (usar 0 en evaluación); *is_causal=True* aplica la máscara triangular sin construirla; *scale* reemplaza el $1/\sqrt{d_k}$ por defecto. No devuelve los pesos.

nn.MultiheadAttention(embed_dim, num_heads, batch_first, kdim, vdim): módulo completo con proyecciones Q/K/V (*in_proj_weight* 3E×E) y de salida. embed_dim debe ser divisible entre num_heads; *batch_first=True* usa (B, n, E); *kdim/vdim* permiten que K y V vengan de otra dimensión (cross-attention). En forward(query, key, value): *key_padding_mask* (B, n_k) ignora el padding, *attn_mask* ($n_q \times n_k$ o $B \cdot h \times n_q \times n_k$) impone p. ej. causalidad, *need_weights* devuelve los pesos y *average_attn_weights* decide si se promedian entre cabezas o se devuelven por cabeza.

nn.TransformerEncoderLayer(d_model, nhead, dim_feedforward, dropout, norm_first): bloque atención + FFN con residuales; *dim_feedforward* es d_{ff}, *norm_first=True* lo convierte en Pre-LN (por defecto Post-LN). **nn.Transformer.generate_square_subsequent_mask(n)** devuelve una matriz float n×n con 0 en y bajo la diagonal y –inf encima: la máscara causal aditiva.

Hugging Face. Con *output_attentions=True* el modelo devuelve *outputs.attentions*: una tupla con **una entrada por capa** (12 en BERT-base y GPT-2), cada una de forma **(batch, cabezas, n, n)**, donde la fila i son los pesos softmax de la query i sobre todas las keys. Requiere *attn_implementation="eager"*, porque SDPA/Flash no materializan los pesos.

Verificaciones de la implementación. Nuestra atención desde cero coincide con `F.scaled_dot_product_attention` (error máx 3.6e-07, con y sin máscara causal); nuestro multi-head con los pesos copiados reproduce `nn.MultiheadAttention` (error 1.2e-07, mismos 4,224 parámetros); y, aplicada a los embeddings de BERT con las proyecciones de su capa 0, reproduce exactamente las atenciones que devuelve el modelo (error 0.0e+00).

4. Visualización de atención en BERT y GPT-2

Para cada cabeza calculamos la masa media de atención hacia: token anterior, siguiente, [CLS]/primer token, [SEP], puntuación y el mismo token, y elegimos la cabeza más representativa de cada patrón en capas distintas (oración: *«The quick brown fox jumps over the lazy dog, and then it falls asleep.»*). En **BERT** (fig. 3) aparecen patrones muy nítidos: L2-H9 atiende al token siguiente (0.94), L3-H5 al anterior (0.67), L1-H6 casi todo a [CLS] (0.90), L7-H3 a [SEP] (0.85) y L11-H0 a la coma y el punto (0.89). La atención a la misma palabra es débil en BERT (máx. 0.23). 54 de 144 cabezas ponen más del 50 % de su masa en [CLS]/[SEP], concentradas en las capas 4–9, igual que reportan Clark et al. (2019). En **GPT-2** (fig. 4) hay una cabeza de token anterior casi perfecta (L4-H11, 1.00), una de la misma palabra en la capa 0 (0.98) y cabezas que mandan prácticamente todo al primer token (L5-H1, 0.99).

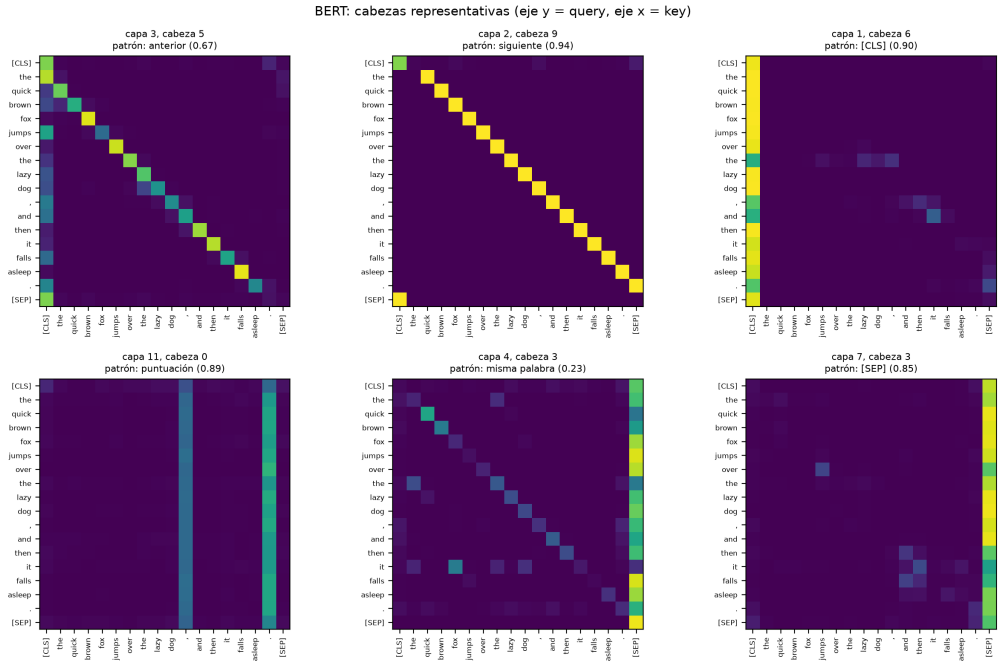


Fig. 3. BERT: seis cabezas de capas distintas (eje y = query, eje x = key) con el patrón detectado y su puntaje.

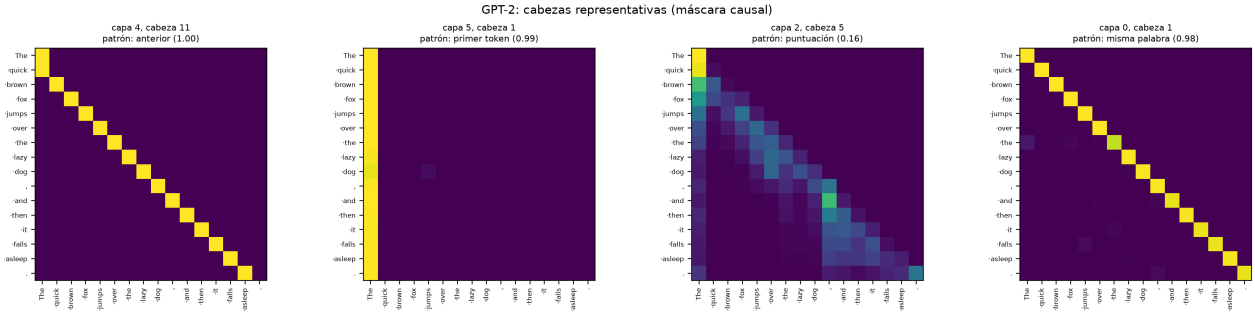


Fig. 4. GPT-2: cabezas representativas. Toda la masa queda en o bajo la diagonal por la máscara causal.

Correferencia de «it». Con «…because it was too *tired*» vs «…too *wide*», buscamos cabezas con a(it→animal) > a(it→street) en el primer caso y lo contrario en el segundo. 22 de 144 cabezas cumplen la condición. La mejor es **capa 6, cabeza 10** (fig. 5): con *tired* «it» atiende a animal 0.28 vs street 0.07, y con *wide* a animal 0.01 vs street 0.59. Las cabezas que cambian de opinión se agrupan en las capas medias-altas (4–10), donde la representación ya es semántica. Sin embargo, promediando todas las cabezas la diferencia es mínima (Δ_{tired}=0.068, Δ_{wide}=0.015): la información está concentrada en pocas cabezas.

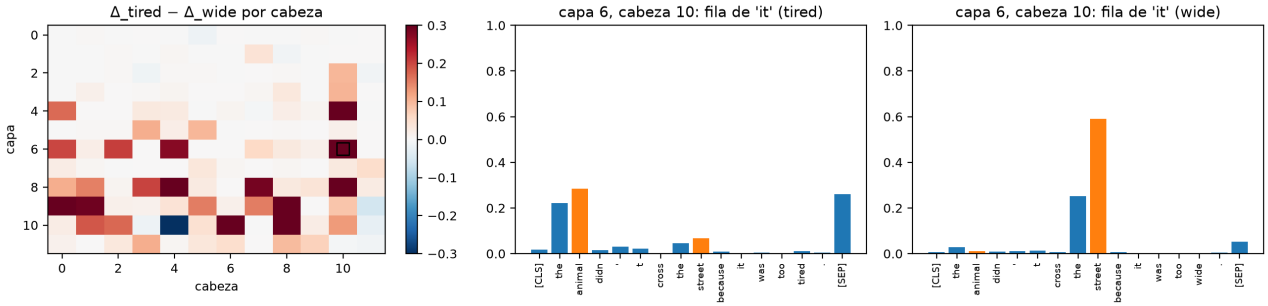


Fig. 5. Izquierda: $\Delta_{tired} - \Delta_{wide}$ por cabeza (rojo = la cabeza cambia de animal a street según el contexto). Derecha: fila de «it» en la mejor cabeza para ambas oraciones.

Máscara causal y attention sinks en GPT-2. Con un texto de 44 tokens, el peso máximo por encima de la diagonal en las 144 cabezas fue 0.0 (exactamente cero): la matriz es exactamente triangular inferior y cada fila suma 1. La proporción media de atención al primer token crece de 0.10 en la capa 0 a 0.75 en las capas medias (fig. 6), muy por encima de los 0.08 esperados si la atención fuera uniforme. Al anteponer «Yesterday» el efecto es idéntico, así que depende de la **posición** y no del contenido del token. Es el fenómeno de **attention sinks** (Xiao et al., 2023): como la softmax obliga a repartir masa 1, las cabezas que «no necesitan» atender a nada vuelcan el sobrante en el primer token, visible para todas las queries; su valor tiene norma pequeña, por lo que actúa como un no-op. Por eso StreamingLLM conserva siempre los primeros tokens en la caché KV: si se descartan, la perplejidad se dispara. BERT hace lo mismo con [SEP] en capas 4–9 y con [CLS] en las primeras capas.

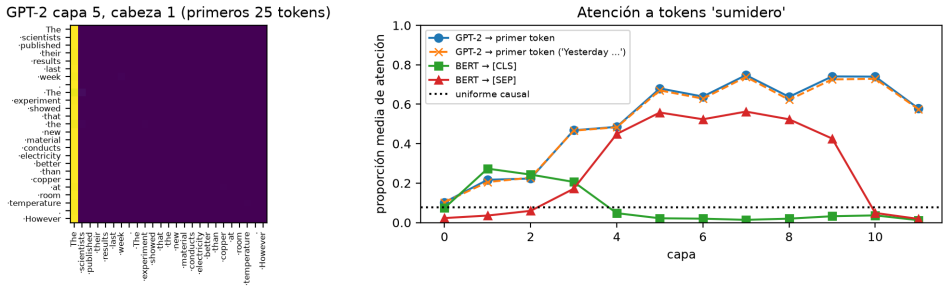


Fig. 6. Izquierda: cabeza «sumidero» de GPT-2. Derecha: masa media de atención al primer token (GPT-2) y a [CLS]/[SEP] (BERT) por capa.

Entropía por capa (fig. 7, promedio de 6 oraciones; en GPT-2 se normaliza por log del número de keys visibles). En ambos modelos las primeras capas son las más difusas (BERT capa 0: 0.77; GPT-2 capa 1: 0.72) y la atención se **concentra** al profundizar: mínimo en la capa 5 de BERT (0.41) y en la capa 10 de GPT-2 (0.31). En GPT-2 la caída es casi monótona, impulsada por las cabezas sumidero; BERT vuelve a difundirse en las últimas capas (≈0.56), que mezclan contexto amplio para la tarea de MLM, y GPT-2 también repunta en la capa 11 (0.45). Así, las capas profundas atienden de forma más concentrada que las iniciales, aunque parte de esa concentración es hacia tokens sin contenido.

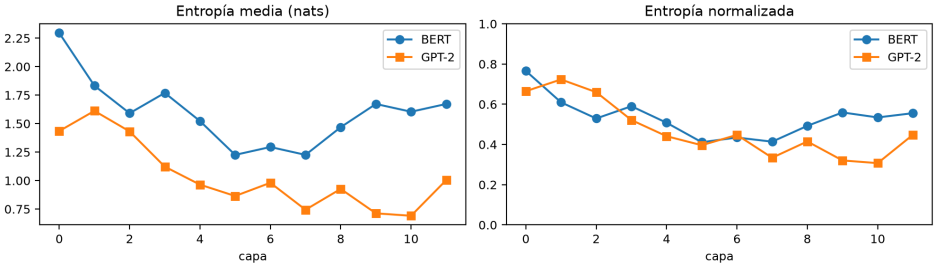


Fig. 7. Entropía media de la atención por capa (nats) y normalizada por su máximo posible.

5. Discusión

¿Por qué la atención permitió modelos mucho más grandes? (1) Paralelismo: todos los tokens se procesan con multiplicaciones de matrices, aprovechando GPUs/TPUs durante el entrenamiento (las RNN requieren n pasos secuenciales). (2) Longitud de camino O(1) entre cualquier par de tokens frente a O(n), lo que hace aprendibles las dependencias largas y el flujo de gradiente estable con residuales y LN. (3) El costo cuadrático es el precio, pero para los contextos de entrenamiento habituales (512–4k tokens) es aceptable y altamente paralelizable, por lo que el cómputo total escala con datos y parámetros (leyes de escalamiento) mejor que con recurrencia.

Especialización de cabezas. Observamos cabezas posicionales casi deterministas (anterior/siguiente), cabezas de puntuación, cabezas de delimitadores y 22 cabezas sensibles a la correferencia. Pero no todas parecen útiles: 54 de 144 cabezas de BERT y la mayoría de las capas medias de GPT-2 vuelcan su masa en [SEP]/[CLS]/primer token. Esto concuerda con Michel et al. (2019) y Voita et al. (2019), que podan gran parte de las cabezas con poca pérdida de desempeño.

¿Los pesos de atención explican las decisiones? Solo parcialmente. Jain y Wallace (2019) muestran que los pesos se correlacionan débilmente con medidas de importancia por gradiente y que existen distribuciones de atención muy distintas que producen la misma predicción. Wiegreffe y Pinter (2019) responden que la atención puede ser *una* explicación plausible si se evalúa bajo condiciones adecuadas, pero no *la* explicación única. Nuestros resultados ilustran ambas posturas: la cabeza L6-H10 sí «sigue» al referente correcto de «it», pero es una de 144, el promedio casi no distingue, y la mayor masa de atención va a tokens sumidero cuyos valores apenas contribuyen a la salida (atención alta ≠ influencia alta). Además ignoran las transformaciones de V, la FFN y los residuales. Para afirmar causalidad harían falta intervenciones (ablación de cabezas, gradientes, probing).

¿Cuándo preferir RNN o CNN? Streaming y dispositivos embebidos con memoria constante por paso (reconocimiento de voz en línea, control, series de tiempo largas): una RNN/LSTM o un modelo de espacio de estados (Mamba, S4) procesa cada token en O(1). Datos pequeños con fuerte estructura local (imágenes pequeñas, señales 1D, audio): las CNN aportan sesgos inductivos de localidad e invariancia a la traslación y requieren muchos menos datos que un ViT. También cuando la latencia o el costo de inferencia en hardware limitado domina.

Documentos de 100,000 tokens. La matriz de atención tendría 10^{10} entradas por cabeza: ≈298 GB en fp32 para una sola capa de 8 cabezas, y el tiempo crece cuadráticamente; además la caché KV de inferencia crece linealmente y los embeddings posicionales absolutos no extrapolan. Alternativas: FlashAttention (atención exacta con memoria lineal) + RoPE con interpolación de posiciones (YaRN); atención por ventana deslizante con tokens globales (Longformer/BigBird) o sumideros (StreamingLLM); atención lineal o modelos de espacio de estados (Mamba) en O(n); o recuperar fragmentos relevantes (RAG / chunking jerárquico) y atender solo sobre ellos.

6. Conclusiones

La atención resuelve el cuello de botella de seq2seq con una búsqueda diferenciable sobre todos los estados y, al eliminar la recurrencia, hace el entrenamiento paralelo y acorta a O(1) el camino entre tokens. Verificamos que nuestra implementación es numéricamente idéntica a PyTorch y a BERT, que el factor $1/\sqrt{d_k}$ es necesario para evitar la saturación de la softmax y que el costo crece cuadráticamente con n. En los modelos preentrenados las cabezas se especializan (posición, puntuación, delimitadores, correferencia), muchas actúan como sumideros y la atención se concentra en capas profundas; los pesos de atención son una herramienta útil de inspección, pero no una explicación causal suficiente del comportamiento del modelo.

Referencias

• Vaswani, A. et al. (2017). Attention Is All You Need. arXiv:1706.03762.
• Bahdanau, D., Cho, K., Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.
• Luong, M.-T., Pham, H., Manning, C. (2015). Effective Approaches to Attention-based Neural Machine Translation. EMNLP.
• Clark, K. et al. (2019). What Does BERT Look At? An Analysis of BERT's Attention. BlackboxNLP.
• Xiao, G. et al. (2023). Efficient Streaming Language Models with Attention Sinks. arXiv:2309.17453.
• Jain, S., Wallace, B. (2019). Attention is not Explanation. NAACL. · Wiegreffe, S., Pinter, Y. (2019). Attention is not not Explanation. EMNLP.
• Michel, P. et al. (2019). Are Sixteen Heads Really Better than One? NeurIPS. · Voita, E. et al. (2019). Analyzing Multi-Head Self-Attention. ACL.
• Xiong, R. et al. (2020). On Layer Normalization in the Transformer Architecture. ICML. · Su, J. et al. (2021). RoFormer (RoPE). arXiv:2104.09864.
• Dao, T. et al. (2022). FlashAttention. NeurIPS. · Beltagy, I. et al. (2020). Longformer. · Katharopoulos, A. et al. (2020). Transformers are RNNs. ICML.
• Documentación de PyTorch (torch.nn.functional.scaled_dot_product_attention, nn.MultiheadAttention, nn.Transformer) y de Hugging Face Transformers.

Repositorio: https://github.com/iancumes/HojaDeTrabajo2DeepLearning